

Programmazione Concorrente e Distribuita

Assignment 1 — Pool Game

Versione a Threads e Versione a Task con Java Executor Framework

1. Analisi del problema

Il problema consiste nella realizzazione di un gioco chiamato "Pool Game", simile al biliardo come logica, con un giocatore umano e un bot. Il tavolo contiene un numero potenzialmente elevato di palle (migliaia), ciascuna soggetta a movimento, attrito, rimbalzo sui bordi e collisioni elastiche con le altre palle. Ai vertici superiori del tavolo sono presenti due buche, l'obiettivo del player è spedire le palle nelle buche tramite la propria palla, oppure imbucare la palla del bot.

1.1 Aspetti rilevanti dal punto di vista concorrente

Il sistema presenta diverse entità che operano in modo naturalmente parallelo e che richiedono una gestione attenta della concorrenza.

Parallelismo nella simulazione fisica

Il calcolo più complesso e per il quale il parallelismo viene in aiuto è la simulazione frame-per-frame: aggiornamento delle posizioni di tutte le palle e soprattutto risoluzione delle collisioni. Con migliaia di palle, questo lavoro è intrinsecamente parallelizzabile poiché ciascuna palla è un'entità indipendente, il movimento di una palla non dipende dal movimento simultaneo delle altre nello stesso frame.

Asincronia dei giocatori

Il giocatore umano interagisce tramite tastiera in modo asincrono: i comandi arrivano dall'Event Dispatch Thread (EDT) di Swing in qualsiasi momento. Analogamente, il bot genera impulsi casuali ogni 3 secondi indipendentemente dal ritmo dei frame. Entrambi devono poter interagire con lo stato condiviso (la Board).

Sincronizzazione con il rendering

Il rendering viene eseguito dall'EDT di Swing, un thread separato e non controllabile direttamente. Il GameEngine deve sincronizzarsi con esso per evitare di avanzare al frame successivo prima che quello corrente sia stato mostrato a schermo, prevenendo così lag visivi.

Stato condiviso e race condition

Le palle rappresentano lo stato condiviso principale: vengono lette e modificate sia dai worker (che si occupano di calcolare il movimento e le collisioni) sia dal GameEngine (per il controllo delle buche e del punteggio) sia dalla View (per il rendering). È necessario garantire che le letture e le scritture sullo stato di ogni palla siano atomiche e consistenti.

Collisioni e partizionamento spaziale

La risoluzione delle collisioni è il problema concorrente più delicato: si è adottata una struttura a griglia con un lock per cella, che consente a worker che lavorano su zone distanti del tavolo di operare in parallelo senza limitarsi, questa suddivisione in griglia inoltre permette di effettuare il check delle collisioni in una regione limitata dello spazio di gioco, il che aumenta notevolmente le performance.

2. Architettura e design

Il sistema è strutturato secondo il pattern Model-View-Controller + package util, per sfruttare separazione tra stato del gioco, logica di simulazione e presentazione visiva.

- Componenti principali

Model

- **Ball:** entità thread-safe tramite metodi synchronized. Incapsula posizione, velocità, raggio, massa e stato (viva/morta). Espone metodi atomici per aggiornamento posizione e velocità, oltre al metodo "setLastHitBy" che serve per il punteggio.
- **Board:** contenitore dello stato globale. Gestisce la lista di palline, la Grid2D e i punteggi. I campi volatili (isGameOver, dt) garantiscono safety senza la necessità di usare metodi synchronized ovunque che sono molto costosi.
- **Grid2D:** struttura di partizionamento spaziale. Divide il tavolo in celle di dimensione fissa (40px) e mantiene un lock separato per cella, abilitando parallelismo nella fase di risoluzione delle collisioni.

Controller (versione a Thread)

- **GameEngine (Thread/Master):** coordina il ciclo frame-per-frame. Aggiorna la griglia, calcola dt, e si sincronizza con i Worker tramite una CyclicBarrier a 3 checkpoint, quando finisce la partita o viene chiusa la finestra di gioco, il GameEngine termina.
- **Worker (Thread):** thread permanente che esegue la fase 1 (movimento) e la fase 2 (collisioni) per il sottoinsieme di celle assegnato. Il numero di Workers è impostabile dal file "Configuration". Ogni worker viene terminato direttamente dal GameEngine.
- **BotController (Thread):** esegue un ciclo infinito che ogni 3000ms applica un impulso casuale alla pallina del bot. E' un thread daemon nel senso che termina con la fine del GameEngine.
- **InputController (Thread):** consumatore del BoundedBuffer di comandi. Si blocca in attesa di comandi dalla View e li esegue sulla Board. Anche questo è un daemon che termina col GameEngine.

Controller (versione a Task)

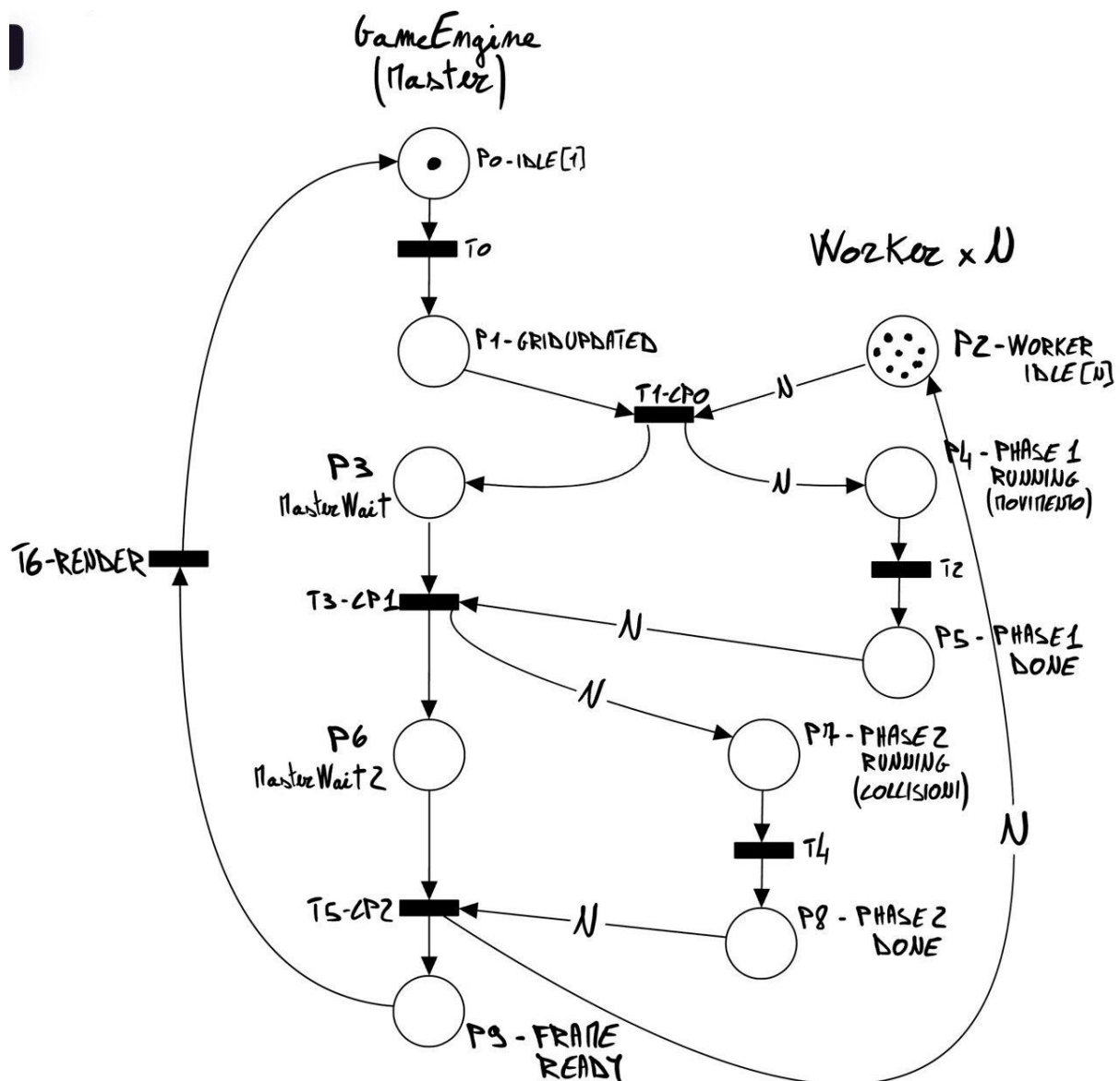
- **GameEngine (Thread/Master):** stessa struttura, ma viene sostituita la CyclicBarrier con due chiamate invokeAll() su un ExecutorService a pool fisso. La barriera implicita di invokeAll() garantisce la separazione tra fase 1 e fase 2, in questo approccio i punti di sincronizzazione sono due e non tre perché i workers non sono più componenti attivi!
- **WorkerTask (Callable<Void>):** task finito che esegue solo la fase 2 (collisioni) per le celle assegnate. La fase 1 è espressa sempre come task, ma come lambda inline in buildMoveTasks().
- **BotController e InputController:** invariati rispetto alla versione a thread, modellano componenti attivi con ciclo infinito, per i quali il task non è il modello appropriato.

View

- ViewFrame: gestisce il rendering sull'EDT e notifica la fine del rendering tramite RenderSynch.
- RenderSynch (Monitor): sincronizza il GameEngine con l'EDT. Il GameEngine ottiene un "ticket" di frame con nextFrameToRender() e attende con waitForFrameRendered() prima di procedere al frame successivo.

3. Comportamento — Rete di Petri

La rete di Petri seguente descrive il comportamento del sistema per la versione a thread. Sono modellati il GameEngine (Master) e N Worker generici, rappresentati come token iniziali nella piazzola P2.



3.2 Transizioni principali

T0: consuma idle, produce gridUpdated. Il Master aggiorna la griglia e il dt, poi si mette in attesa al CP0.

T1 — CP0 (tutti pronti): attivata quando Master + N Worker hanno raggiunto il checkpoint 0. Il Master ha già aggiornato la griglia; i Worker possono iniziare la fase 1.

T2: ogni Worker esegue il movimento delle proprie palline e avanza su P5.

T3 — CP1 (tutti fine fase 1): attivata quando tutti i worker hanno finito la fase 1. Nessun Worker può iniziare la fase 2 prima che tutti abbiano finito la fase 1.

T4: ogni Worker risolve le collisioni per le proprie celle e avanza su P8.

T5 — CP2 (tutti fine fase 2): Il Master riprende il controllo ed effettua altre operazioni che ho deciso di non includere per non rendere la rete di Petri troppo verbosa, i worker tornano in stato idle.

T6: il frame viene renderizzato e il master torna in stato idle, pronto per il frame successivo.

4. Test di performance

Le misurazioni sono state effettuate su una macchina con 12 core fisici e 24 thread logici, con pc non connesso all'alimentazione per garantire uniformità delle condizioni. La metrica adottata è il tempo medio di computazione per frame in millisecondi, escludendo il tempo di rendering (`view.display()` e `waitForFrameRendered()`) per isolare il contributo della logica concorrente.

Sono state testate due configurazioni di gioco: Large (qualche centinaio di palline, carico leggero) e Massive (4502 palline: 4500 regolari + player + bot, carico pesante). I test con carico pesante sono quelli significativi per valutare l'effettivo guadagno della parallelizzazione.

4.1 Configurazione Large

Versione	N_WORKERS	Tempo medio/frame (ms)	Frame totali
Sequenziale	—	0,39	5433
Threads	5	0,35	4272
Threads	13	0,49	4516
Threads	24	0,55	4964
Tasks	5	0,32	5444
Tasks	13	0,31	5541
Tasks	24	0,38	5197

Con carico leggero i risultati sono poco significativi: il tempo per frame è nell'ordine di frazioni di millisecondo e l'overhead di coordinazione tra thread (barriere, `invokeAll`) domina sul tempo di calcolo effettivo. Si nota infatti che aumentare i worker peggiora le performance nella versione a thread, mentre la versione a task mantiene tempi più stabili grazie alla minore overhead dell'Executor rispetto alla `CyclicBarrier`.

4.2 Configurazione Massive

Versione	N_WORKERS	Tempo medio/frame (ms)	Frame totali
Sequenziale	—	41,20	404
Threads	5	10,19	971
Threads	13	8,98	943
Threads	24	9,08	906
Tasks	5	9,74	1032
Tasks	13	9,43	1075
Tasks	24	9,73	804

Con carico pesante il vantaggio della parallelizzazione è evidente. La versione sequenziale impiega 41,20 ms per frame, rendendo il gioco quasi ingiocabile. Le versioni concorrenti raggiungono tempi nell'ordine di 9-10 ms, con un fattore di speedup di circa 4-4,5×.

4.3 Speedup e scalabilità

Versione	N_WORKERS	Speedup vs Sequenziale	Efficienza (su 12 core fisici)
Threads	5	~4,04×	~81%
Threads	13	~4,59×	~35%
Threads	24	~4,54×	~19%
Tasks	5	~4,23×	~85%
Tasks	13	~4,37×	~34%
Tasks	24	~4,23×	~18%

Lo speedup massimo osservato è circa 4,59× con 13 thread nella versione a thread, e 4,37× con 13 task nella versione a task. Questi valori sono significativamente inferiori allo speedup teorico massimo per le seguenti ragioni:

- Ogni pallina è protetta da synchronized. Con migliaia di collisioni per frame, la contention sui lock delle palline rappresenta un collo di bottiglia significativo.
- Overhead di coordinazione: la CyclicBarrier (versione thread) e invokeAll() (versione task) introducono latenza di sincronizzazione ad ogni frame, pagata tre volte per frame nella versione thread e due nella versione task.
- Partizionamento non uniforme: la distribuzione ciclica delle celle può creare sbilanciamento se alcune zone del tavolo hanno molte più palline di altre (es. all'inizio del gioco quando tutte le palline sono concentrate al centro).
- Limite hardware: il test è stato eseguito con laptop non alimentato, inoltre per architettura della CPU, l'efficienza complessiva sui 12 core è limitata (spiegato meglio in seguito).

4.4 Confronto versione Thread vs Task

Le due versioni mostrano performance comparabili sul carico pesante (differenza < 5%). La versione a task tende a fare leggermente meglio sul carico leggero grazie a due fattori: primo, `invokeAll()` introduce meno overhead rispetto alla `CyclicBarrier` a 3 checkpoint (la versione task usa 2 punti di sincronizzazione per frame invece di 3); secondo, l'Executor riusa i thread del pool senza ricreazione, mentre la `CyclicBarrier` ha costo di attesa e risveglio ad ogni checkpoint.

Con `N_WORKERS` = 24 (superiore al numero di core fisici) entrambe le versioni mostrano un leggero peggioramento rispetto a `N_WORKERS` = 13, confermando che l'iper-threading non porta benefici per carichi CPU-bound come questo, e che il context switching aggiuntivo diventa un overhead netto.

La misurazione con laptop alimentato (versione Thread, 13 worker, Massive) mostra 7,52 ms rispetto agli 8,98 ms a batteria, confermando che la limitazione energetica del processore impatta le performance di circa il 16%.

4.5 Analisi dell'utilizzo CPU — AMD Ryzen AI 9 HX 370

La macchina di test monta un processore AMD Ryzen AI 9 HX 370 che presenta caratteristiche rilevanti per interpretare correttamente i risultati di speed-up ed efficienza osservati.

Architettura ibrida Zen 5 / Zen 5c

Il processore adotta un'architettura ibrida con due tipologie di core fisicamente distinte:

- 4 core Zen 5 (Performance): clock fino a 5.1 GHz.
- 8 core Zen 5c (Efficiency): clock massimo 3.3 GHz.

Totale: 12 core fisici, 24 core virtuali. Questo spiega perché con `N_WORKERS` = 24 le performance non migliorano rispetto a 13: i thread in eccesso vengono schedati su core fisici già occupati visto che i core virtuali si appoggiano su quelli fisici per svolgere calcoli, introducendo latenza senza aggiungere potenza di calcolo reale.

Perché `N_WORKERS` = 13 è il punto ottimale

Il valore `N_WORKERS` = 13 corrisponde al numero di core fisici (12) più uno, il classico suggerimento per carichi CPU-bound. Con 13 worker il pool occupa tutti i core fisici disponibili lasciando risorse per il GameEngine (Master), il BotController, l'InputController e l'EDT di Swing. Con `N_WORKERS` = 24 il numero di thread è maggiore di quanti core fisici possano servire contemporaneamente, il che introduce latenza che per un carico CPU-bound come la simulazione fisica non è ottimale (anche se comunque le performance sono molto vicine).

Utilizzo CPU osservato — Monitoraggio Risorse

Le immagini seguenti mostrano l'utilizzo della CPU durante una sessione di gioco con configurazione Massive e `N_WORKERS` = 13.

Processi							Utilizzo CPU 23%	100% frequenza massima
Processo	PID	Descrizione	Stato	Thread	CPU	Media CPU		
☒ java.exe	27064	OpenJDK Platform binary	In esecuzione	85	9	8.13		
☐ Secure System	332		Sospeso	-	0	0.00		
☐ ShellExperienceHost.exe	9300	Windows Shell Experience Host	Sospeso	43	0	0.00		
☐ SystemSettings.exe	10356	Impostazioni	Sospeso	67	0	0.00		
☐ LockApp.exe	3896	LockApp.exe	Sospeso	35	0	0.00		
☐ backgroundTaskHost.exe	10024	Background Task Host	Sospeso	8	0	0.00		
☐ WmiPrvSE.exe	5952	WMI Provider Host	In esecuzione	10	1	0.87		
☐ Interrupt sistema	-	Chiamate di procedura differite e ISR (Interrupt Service Routine)	In esecuzione	-	0	0.51		

Figura 1 — Il processo java.exe con 85 thread attivi, CPU istantanea al 9%, media 8.13. La CPU opera alla frequenza massima (100% di 5.1 GHz).

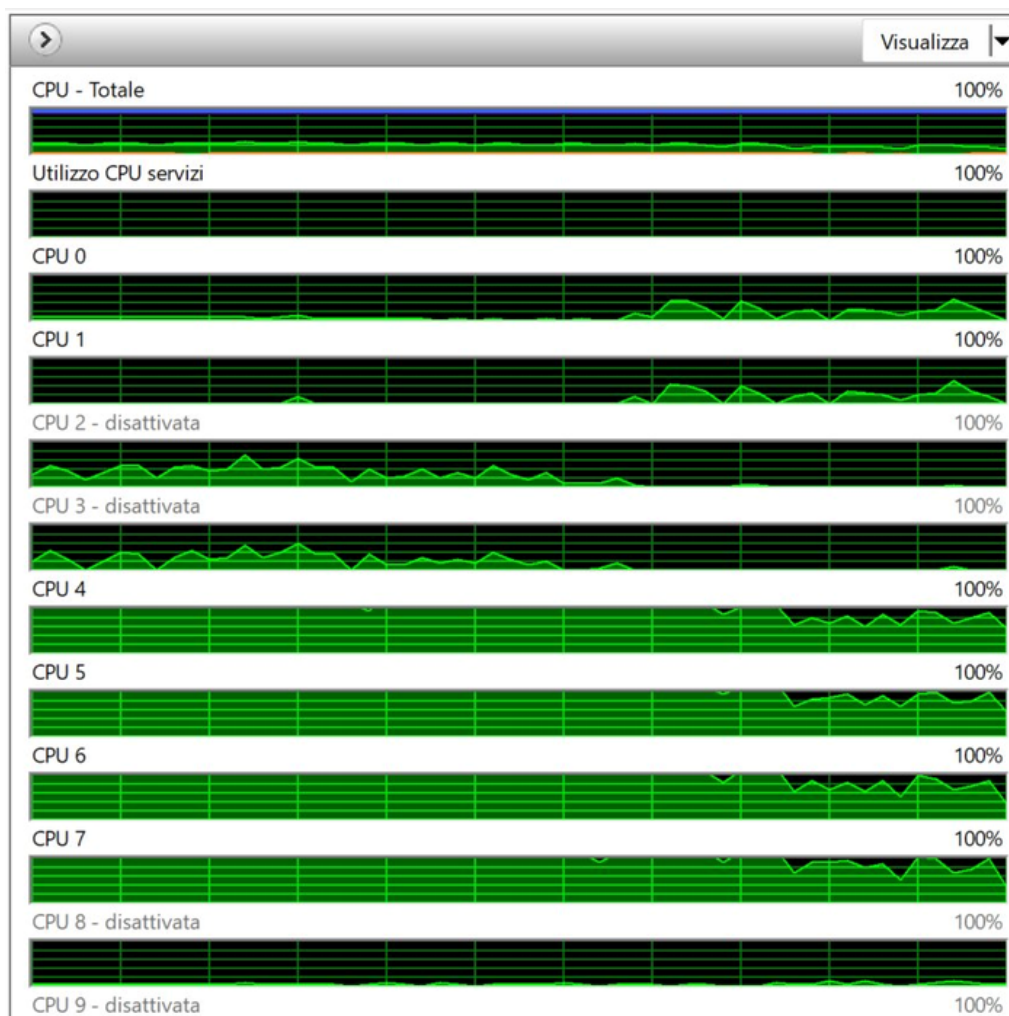


Figura 2 — Grafici per core: 4 core al 100% quasi costante, gli altri a utilizzo variabile.

I grafici confermano il comportamento atteso: la CPU non appare saturata nel grafico totale (~23%) perché ogni frame alterna una fase di calcolo (~9 ms, core attivi al 100%) e una fase di attesa del rendering da parte dell'EDT. Inoltre, per architettura della CPU, invece che distribuire il calcolo su tutti i core, per un discorso di efficienza energetica, il calcolo viene schedato sui 4 core performance, lasciando gli altri più scarichi. La media risultante è strutturalmente bassa pur in presenza di un utilizzo intenso durante la fase computazionale.

5. Note implementative rilevanti

5.1 CyclicBarrierImpl con campo generation

Il campo "generation" introdotto nella barriera permette di resettarsi automaticamente, rendendola così ciclica, senza la necessità di dover creare più istanze: il campo generation è un po' come se fosse il turno corrente, i thread che arrivano si mettono in attesa se `currentGeneration == generation`, tranne l'ultimo che vede `nArrived == nParticipants`, ed entra nell'if, azzerando `nArrived` per il setup della barriera per il turno successivo, incrementa il numero del turno e sveglia tutti, i thread svegliati vedono che la condizione di attesa non è più rispettata e partono, la barriera è già pronta per il turno successivo. Questo concetto di fare il check sul numero del turno nel while e non per esempio su `nArrived < nParticipants` è fondamentale per una barriera ciclica senno' avverrebbe deadlock sicuramente!! Un thread veloce bloccato sulla wait del turno 2 e tutti gli altri bloccanti sulla wait del turno 1 e nessuno che incrementerebbe più `nArrived`.

5.2 Ottimizzazione del calcolo delle collisioni

Il rilevamento delle collisioni usa due ottimizzazioni: un pre-check che evita il calcolo della distanza quando le palline sono certamente lontane, e l'uso della distanza al quadrato rispetto alla `minDistance` al quadrato, rimandando il **calcolo della radice quadrata solo ai casi di collisione effettiva**. Con migliaia di palline e decine di frame al secondo, questo riduce significativamente il numero di radici calcolate.

5.3 RenderSynch come monitor

Il monitor `RenderSynch` usa un contatore di frame, garantendo che il `GameEngine` non avanzi se l'EDT è in ritardo di più di un frame. La scelta elimina completamente la possibilità di accumulo, al costo di legare il ritmo del calcolo a quello del rendering, questa scelta però di non accodare frame pronti nel buffer dell'EDT riduce notevolmente l'input lag (reattività dei comandi dati da tastiera per muovere la pallina), che nei giochi è sempre un punto fondamentale da tenere in considerazione.

5.4 Terminazione pulita

La chiusura della finestra imposta `board.setGameOver(true)`, che fa uscire il `GameEngine` dal ciclo while in automatico. Il finally del `GameEngine` stampa le statistiche di performance, termina i worker thread (l'executor nell'approccio a task) e chiama `System.exit(0)` per garantire la terminazione di tutti i thread daemon rimasti attivi (`BotController`, `InputController`, EDT di `Swing`).

6. Verifica con model checking (JPF)

Il model checking è stato condotto tramite Java PathFinder (JPF) sui due monitor implementati e sul monitor di sincronizzazione con il rendering: `CyclicBarrierImpl`, `BoundedBufferImpl` e `RenderSynch`. La scelta di limitare la verifica a questi tre componenti è

motivata dal fatto che rappresentano tutta la logica di sincronizzazione del sistema, tutto il resto si appoggia a meccanismi primitivi della JVM (synchronized, volatile).

Per tutti i test è stato abilitato il listener `PreciseRaceDetector`, che verifica sia l'assenza di deadlock sia l'assenza di race condition sullo stato condiviso.

TestBarrier simula 3 thread che si sincronizzano su `CyclicBarrierImpl` per 2 round consecutivi (ho messo questi parametri per evitare un'attesa troppo lunga, ma il ragionamento è lo stesso con N thread e K round), coprendo il caso critico del reset automatico della barriera tramite il campo `generation`. JPF ha esplorato 1675 stati distinti con profondità massima 63, esplorando tutti gli scenari possibili senza rilevare errori.

TestBoundedBuffer simula un produttore e un consumatore su un buffer di capacità 2, con 3 operazioni di put e 3 di get, forzando sia il caso di buffer pieno (produttore in attesa) sia il caso di buffer vuoto (consumatore in attesa). JPF ha esplorato 846 stati con profondità massima 79, senza rilevare errori.

TestRenderSynch simula il `GameEngine` e l'EDT come due thread ordinari su 3 frame consecutivi, verificando che il `GameEngine` non possa mai avanzare al frame N+1 prima che l'EDT abbia completato il rendering del frame N tramite `notifyFrameRendered()`. Poiché JPF non modella Swing, l'EDT è stato sostituito con un thread normale che chiama esclusivamente i metodi di `RenderSynch`. JPF ha esplorato 576 stati con profondità massima 45, senza rilevare errori.

In tutti e tre i casi JPF ha verificato l'assenza di deadlock e race condition su tutti i possibili ordinamenti di esecuzione dei thread, confermando la correttezza dei monitor implementati negli scenari testati.